Serial input needs real clock edges

From the original for-loop mistake to one-bit-per-edge capture, complete frames and odd-parity timing.

Kapil Tripathi | Verilog & digital design | 11 pages

CONTENTS / click a row to jump

KEEP THIS IDEA

A loop without timing control runs within one process activation. A bit counter preserves progress between clock edges.

Tracker entries: 2, 4, 6

Study notes by Kapil Tripathi, based on HDLBits exercises. Original questions, source references and dated verification records are retained in the notes.

ALL NOTES + ENTRY INDEX HDLBits problem collection

1. Start from the clock sequence

The base receiver detects a start bit, waits for eight data bits, and validates the stop bit. The datapath version then preserves those eight sampled bits so that `out_byte` is valid when `done` is asserted. HDLBits explicitly says the serial stream is least-significant-bit first and is shifted in one bit at a time. [1][2]

Problem	What changes	Timing responsibility
Fsm_serial	Control only	State remembers start, 8 data clocks, stop, and framing errors.
Fsm_serialdata	Add byte datapath	A counter or shift register records exactly one data bit per clock.
Fsm_serialdp	Add parity bit and checker	The parity accumulator advances on the same serial clock sequence. [3]

The attempted loop

```
for (i = 1; i < 8; i = i + 1) begin
    count <= count + 4'd1;
    data_out[i] <= in;
end
```

> **Why the loop cannot wait**
>
> The loop variable `i` changes during one execution of the process. No `@(posedge clk)`, delay, or state transition appears inside the loop, so nothing advances simulation time and nothing waits for another clock edge.

What synthesis sees

AMD Vivado documents `for` and `repeat` as supported ways to describe repetitive or bit-slice structures inside an `always` block. [4] For constant loop bounds, the hardware meaning is a fixed amount of replicated logic, not a hidden clock sequencer.

2. One rising edge, seven loop iterations

Assume the pre-edge values are `count = 3` and `in = 1`. The clock event wakes the `always @(posedge clk)` process once. The process then executes all seven iterations before the next clock event can occur.

Iteration	RHS read for count	Scheduled bit write
i = 1	old count 3 -> 4	data_out[1] <= 1
i = 2	old count 3 -> 4	data_out[2] <= 1
i = 3	old count 3 -> 4	data_out[3] <= 1
...	same old count	same sampled input
i = 7	old count 3 -> 4	data_out[7] <= 1

Why nonblocking assignment matters

- AMD states that a nonblocking assignment evaluates its expression when the statement executes, while the variable changes later. [5]

- Therefore each `count <= count + 1` evaluates from the same pre-update `count` in this process activation.

- All seven scheduled values are identical here. The observable result after the edge is `count = 4`, not 10.

- The seven different bit-select targets are all scheduled, so `data_out[7:1]` becomes seven copies of the one sampled input value.

> **How the loop runs**
>
> Nonblocking assignment is not a mechanism for repeating work on later clocks. It separates evaluation from update within the current simulation time slot. Clock-to-clock behavior still requires stored state that changes once per clock.

IEEE 1800-2023 is the active SystemVerilog language standard defining the language's syntax and semantics, including RTL and testbench behavior. [6] AMD UG901 supplies the synthesis-oriented interpretation used here. [4][5]

3. Spatial repetition is not temporal repetition

Question	Spatial repetition	Temporal repetition
What repeats?	Hardware operations or bit-slice assignments	A registered action across clock edges
What advances it?	Loop variable during one process activation	Counter, FSM state, shift register, or handshake
How many input samples?	One sample for this clock event	One sample on each selected future clock
Typical RTL	for loop over vector bits	if (enable) count <= count + 1
UART byte capture	Wrong abstraction for consuming the stream	Exactly the required abstraction

A useful hardware picture

SPATIAL - same cycle

```
a[0] & b[0] -> out[0]
a[1] & b[1] -> out[1]
a[2] & b[2] -> out[2]
...
a[7] & b[7] -> out[7]
```

TEMPORAL - different cycles

```
edge 1: in -> data_out[0]
edge 2: in -> data_out[1]
edge 3: in -> data_out[2]
...
edge 8: in -> data_out[7]
```

Fast diagnostic question

Ask: 'What register tells the circuit which byte bit this clock belongs to?' If the answer is only the loop variable, the design has no clock-to-clock memory for that position.

4. Local simulation evidence

A small self-checking Icarus Verilog simulation was generated for this document. The first DUT contains the original loop. The second DUT uses one counter-controlled write per edge. The testbench fails immediately if either result differs from the expected clock semantics.

```
BAD one edge: count=1 data_out=11111110
GOOD eight edges: count=0 data_out=10100110
PASS: spatial loop and temporal counter behaviors verified
```

Interpreting the result

Observed result	Meaning
BAD count = 1	Seven identical nonblocking increment values were computed from old count = 0.
BAD data_out = 11111110	Bits 1 through 7 copied the single input value 1; bit 0 was untouched.
GOOD count = 0	Eight real rising edges occurred and the 3-bit position counter wrapped after bit 7.
GOOD data_out = 10100110	The LSB-first stream was reconstructed into the intended parallel byte.

What was checked

This test checks the loop and counter examples shown here. The complete original serial-receiver submission is not saved locally, so this is not a rerun of that submission.

5. Correct RTL capture patterns

Pattern A - indexed write with a registered bit counter

```
always @(posedge clk) begin
    if (reset) begin
        count    <= 3'd0;
        data_out <= 8'd0;
    end else if (state == DATA) begin
        data_out[count] <= in;

        if (count == 3'd7)
            count <= 3'd0;
        else
            count <= count + 3'd1;
    end else begin
        count <= 3'd0;
    end
end
```

At each data-state edge, the old registered `count` selects one destination bit. The nonblocking update then prepares the next count for the next edge. This is precisely the old-value behavior the design needs.

Pattern B - shift register

```
always @(posedge clk) begin
    if (reset)
        shift_reg <= 8'd0;
    else if (state == DATA)
        shift_reg <= {in, shift_reg[7:1]};
end
```

Because HDLBits transmits the least-significant bit first, shifting right and inserting the newest bit at bit 7 reconstructs the final byte after eight data clocks. [2] Either indexed capture or shifting is valid; use one consistent convention and verify it with a known asymmetric byte.

> **Off-by-one checkpoint**
>
> Decide whether `count` names the bit being captured now or the number already captured. Then keep state transitions, final-bit capture, stop-bit validation, and `done` aligned to that definition.

6. Eight-clock trace

Use the test byte `8'b10100110`. Since the protocol is LSB-first, the serial samples are 0, 1, 1, 0, 0, 1, 0, 1. The table shows values after each edge.

Edge	Old count	in	Write	Count after	data_out after
1	0	0	bit 0	1	xxxxxxx0
2	1	1	bit 1	2	xxxxxx10
3	2	1	bit 2	3	xxxxx110
4	3	0	bit 3	4	xxxx0110
5	4	0	bit 4	5	xxx00110
6	5	1	bit 5	6	xx100110
7	6	0	bit 6	7	x0100110
8	7	1	bit 7	0	10100110

Why the last row is safe

- The left-hand bit-select uses the old count value 7 for this edge.
- The count update to 0 occurs later in the nonblocking update phase.
- The completed byte can be consumed during the stop/done phase after the eighth data bit has already been stored.

> **Points to remember**
>
> A counter does not merely count; it is the circuit's memory of bit position. In this receiver, that position selects both what the current input means and where it must be stored.

7. Link the experience to the problem series

HDLBits problem	Experience link	Revision prompt
Fsm_serial [1]	State is time: start -> 8 data clocks -> stop/error.	Can the FSM recover if the stop bit is missing?
Fsm_serialdata [2]	Main lesson: one input sample and one byte-position update per data edge.	Can you trace an asymmetric byte LSB-first?
Fsm_serialdp [3]	Extend the same temporal chain by one parity-bit clock; reset parity at the frame boundary.	Which edge resets parity and which edge tests it?
Fsm_hdlc	Related Day 1 lesson: registered history and output phase matter more than procedural-looking code.	Does the output describe this input now or a remembered sequence?

The three linked exercises

The tracker links entries 2, 4 and 6 to this PDF. They share the same capture sequence: a start bit, eight data edges, and a stop check. Entry 6 adds a parity edge before the stop bit. Pages 10-11 show that timing.

Progress and local checks

The tracker records your progress. The local tests check the examples saved here. A passing example is useful for understanding the fix, but it does not mean every historical submission was rerun.

8. Debugging checklist and references

Check	Question to ask
Clock contract	How many rising edges are required by the protocol?
State memory	Which register remembers where the design is between edges?
NBA old value	Which expressions read pre-edge register values?
Counter meaning	Does count mean current index or completed-bit count?
Bit order	Is the stream LSB-first, and does the shift/index convention match?
Final data bit	Is bit 7 stored before stop-bit validation and done?
Parity extension	Is parity reset once per frame and tested after all 9 bits?
Evidence	Is there a success screenshot, source file, or self-checking simulation?

References

[1] HDLBits, 'Fsm serial'. https://hdlbits.01xz.net/wiki/Fsm_serial
Protocol framing, 8 data-bit clocks, stop-bit recovery, synchronous reset. Accessed 30 August 2026.

[2] HDLBits, 'Fsm serialdata'. https://hdlbits.01xz.net/wiki/Fsm_serialdata
One-bit-at-a-time datapath, parallel byte output, LSB-first ordering. Accessed 30 August 2026.

[3] HDLBits, 'Fsm serialdp'. https://hdlbits.01xz.net/wiki/Fsm_serialdp
Odd parity, ninth serial bit, provided parity accumulator, done gating. Accessed 30 August 2026.

[4] AMD, Vivado Design Suite User Guide: Synthesis (UG901), 'For and Repeat Statements,' 2026.1.
https://docs.amd.com/r/en-US/ug901-vivado-synthesis/For-and-Repeat-Statements
Synthesis use of loops for repetitive and bit-slice structures in always blocks. Accessed 30 August 2026.

[5] AMD, UG901, 'Using Blocking and Non-Blocking Procedural Assignments,' 2026.1.
https://docs.amd.com/r/en-US/ug901-vivado-synthesis/Using-Blocking-and-Non-Blocking-Procedural-Assignments
RHS evaluation at statement execution and deferred variable update. Accessed 30 August 2026.

[6] IEEE Standards Association, IEEE 1800-2023. https://standards.ieee.org/ieee/1800/7743/
Active SystemVerilog language standard defining syntax and semantics. Accessed 30 August 2026.

Prepared from Kapil's pasted discussion. Technical claims were checked against the three HDLBits problem statements, AMD synthesis documentation, the active IEEE standard record, and a local self-checking Icarus Verilog simulation.

9. Follow a complete serial frame

Entries 2 and 4: start, data and stop

While idle, a sampled 0 starts a frame. That edge is the start bit, so it must not also write data bit 0. Capture bit 0 on the next rising edge and bit 7 on the eighth data edge. The following edge checks the stop bit.

Edge	Input meaning	Action
Start	0	Enter DATA; set bit index to 0
Next 8 edges	b0 through b7, LSB first	Capture one bit per edge
Stop	Must be 1	Pulse done if the frame is valid
Following edge	Possible next start bit	Return to start detection without losing a back-to-back frame

What happens when the stop bit is missing?

If the stop sample is 0, suppress done and wait for a sampled 1. Zeros during this recovery period are still part of the malformed frame; do not treat each one as a new start. Once a 1 has restored idle framing, a later sampled 0 may start a new frame.

When can out_byte be read?

Read it while done is high. The eight writes have finished before the stop edge, so the assembled byte is ready when that edge validates the frame. The exercise does not require out_byte to be meaningful while done=0.

The x values on page 7 mark positions that have not yet been captured in that example. With the reset-to-zero code on page 6, those positions would contain zero instead. The bit order and final byte are the same.

Sources: https://hdlbits.01xz.net/wiki/Fsm_serial and https://hdlbits.01xz.net/wiki/Fsm_serialdata

10. Add odd parity at the right edge

Entry 6: one more sample before the stop bit

Odd parity means the XOR of the eight data bits and the parity bit is 1. Reset the parity accumulator at the start boundary, include all eight data samples, and include the parity sample. The stop bit must not affect the value used for this check.

```
// Meaning: parity_total contains the XOR accumulated so far.
// Clear it before data bit 0. During DATA and PARITY:
parity_total <= parity_total ^ in;
// At the following STOP edge, the old value includes parity:
// valid_frame = in && parity_total;
```

The snippet shows the timing rule. In the HDLBits exercise, connect the provided parity module and choose its reset timing to implement that rule. Its accumulator toggles on each sampled 1, so inspect its old output at the stop edge before that edge can include the stop bit.

Stage just completed	What the stored XOR contains
Start	0
Data bit 0	b0
Data bit 7	b0 XOR b1 XOR ... XOR b7
Parity bit	All eight data bits XOR parity
Stop test	Use the stored nine-bit XOR; require stop=1

For byte A6, the data has four ones. A parity bit of 1 makes five, so it is valid. A parity bit of 0 makes four, so done must stay low. In both cases a valid stop bit ends the frame. A zero stop bit uses the framing-error recovery from page 10.

If you check parity_total in the same block activation that captures the parity bit, it still contains only the eight data bits. Either test parity_total ^ in at that edge, or wait until the stop edge and test the stored value. Mixing these two conventions causes a one-bit timing error.

Source: https://hdlbits.01xz.net/wiki/Fsm_serialdp